VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

CO3037: Internet of Things Application Development

Assignment Report

YoloUNO-PlatformIO IoT System

Advisor(s): Lê Trọng Nhân
Phan Văn Sỹ
Student(s): Student 1: Phạm Gia Khiêm - 2352545
Student 2: Hồ Trần Minh Đạt - 2352227
Student 3: Nguyễn Hiếu - 2352327

HO CHI MINH CITY, DECEMBER 2025

Contents

List of Figures

List of Tables

Listings

1 Introduction

1.1 Team member

Full name	Student ID	Tasks	Contribution
Phạm Gia Khiêm	2352545	Task 1 + 6	100%
Hồ Trần Minh Đạt	2352227	Task 3 + 4	100%
Nguyễn Hiếu	2352327	Task 2 + 5	100%

1.2 Background and Motivation

The world is currently undergoing a profound digital revolution that is transforming the way societies function, how businesses operate, and the manner in which individuals engage with technology in their everyday lives. At the core of this revolution is the rise of the Internet of Things (IoT), a paradigm that connects physical devices—ranging from simple household ob jects to sophisticated industrial machines—to the digital world. These connections allow for the continuous flow of data, automation of tasks, and the creation of smart environments that respond adaptively to changes in real time.

Over the past decade, IoT has rapidly shifted from a futuristic concept to a tangible reality, with billions of devices now interconnected globally. The growing affordability of microcon troller units (MCUs), advances in sensor technologies, and the availability of powerful cloud computing resources have been instrumental in democratizing access to IoT solutions. The con vergence of these technologies has fostered an ecosystem where even small teams and individu als can build robust, scalable, and intelligent systems capable of addressing complex challenges.

IoT has found widespread application across diverse fields. In smart homes, IoT enables seamless control of lighting, heating, cooling, and security, enhancing comfort, safety, and energy efficiency. In agriculture, IoT-driven precision farming techniques help farmers mon itor soil moisture, optimize irrigation, and manage crops more efficiently, contributing to in creased productivity and sustainability. In healthcare, IoT devices facilitate continuous patient monitoring, support early diagnosis through data analytics, and enable remote healthcare deliv ery—redefining patient care and improving health outcomes. The manufacturing sector benefits from IoT through predictive maintenance, process automation, and real-time supply chain man agement, resulting in reduced downtime and increased operational efficiency.

Despite its transformative potential, the development of practical, scalable IoT solutions remains a complex endeavor. Designers must address challenges related to heterogeneous hard ware integration, reliable and secure data acquisition, efficient and robust communication between devices, scalable data storage and processing, and the creation of intuitive user interfaces. Additionally, as the number of connected devices grows, so too does the importance of cyber security and data privacy, making these critical considerations in system design.

A significant trend in contemporary IoT development is the integration of artificial intelli gence (AI) and machine learning (ML) capabilities. The fusion of AI and IoT—often referred to as Artificial Intelligence of Things (AIoT)—enables systems to analyze large volumes of sensor data, detect patterns, make predictions, and autonomously respond instantaneously to user about the weather changes. This shift from simple data collection to intelligent decision making unlocks new possibilities for automation, efficiency, and value creation.

1.3 Project Overview

In response to the aforementioned trends and challenges, this project undertakes the design and implementation of a comprehensive IoT system that leverages both hardware and software advancements. The aim is to deliver a smart, adaptable, and scalable solution for real-time monitoring, control, and analytics, suitable for deployment in various real-world scenarios.

The project adopts a modular and layered architecture, with each layer responsible for spe cific functions and designed to maximize system flexibility and ease of maintenance. The key components of the system include:

- **Device Layer (MCU+Sensors):** The foundational layer comprises microcontroller units connected to a suite of environmental sensors, such as temperature and humidity sensors. This layer is responsible for gathering real-world data, preprocessing it when appropriate, and securely transmitting it for centralized analysis. The system supports straightforward integration of additional sensors and over-the-air (OTA) firmware updates, ensuring that the platform remains adaptable and future-proof.

- **Edge Device:** Situated between the device and higher-level system layers, the edge device aggregates sensor data and performs local computations, such as filtering, data compres sion, or preliminary analysis. By offloading certain tasks from the

cloud, the edge device reduces network latency and bandwidth usage, contributing to a more responsive and efficient system.

- **Dashboard and Visualization:** To empower users with actionable insights, the system features web-based dashboards that present real-time and historical data in a visually en gaging manner. Users can monitor device status, configure profiles, customize their dash boards, and receive critical notifications. The dashboards are designed to be intuitive, enabling users of varying technical backgrounds to interact effectively with the system.

- **Artificial Intelligence Module:** Building upon the data collected, AI modules are inte grated to perform advanced analytics, such as anomaly detection, predictive maintenance and trend analysis. These capabilities not only enhance the system's intelligence but also automate responses to abnormal conditions, thereby improving reliability and safety.

- **Web Application:** Recognizing the importance of accessibility and real-time interaction, the system includes a comprehensive web application. This web-based platform enables users to monitor and control connected devices remotely, visualize real-time and historical data, configure system parameters, and receive timely notifications through any modern web browser. The responsive design ensures seamless user experience across desktops, laptops, tablets, and smartphones, allowing users to interact with the system anywhere and anytime without the need for installing dedicated software.

The system's design prioritizes modularity and scalability, allowing it to be adapted to differ ent application domains with minimal modifications. This flexibility is achieved by abstracting hardware interfaces, adopting standardized communication protocols, and implementing con figurable software modules.

1.4 Objectives

The overarching objectives of this project are:

1. To engineer and deploy a resilient IoT infrastructure using a modular architecture, ensuring effortless integration of diverse hardware and software modules to guarantee future extensibility.

2. To establish reliable, low-latency protocols for secure data acquisition, intelligent processing at the edge and in the cloud, and validated transmission integrity across all system layers.

3. To craft highly intuitive and responsive web-based dashboards and complementary mobile interfaces, designed for streamlined real-time monitoring, comprehensive data visualization, system parameter configuration, and proactive alert notification management.

4. To seamlessly incorporate advanced Artificial Intelligence (AI) and Machine Learning (ML) models to facilitate intelligent data analytics, enabling predictive anomaly detection and the implementation of automated, autonomous operational response mechanisms.

5. To ensure the platform exhibits high-availability, scalability, and ease of maintenance, allowing for rapid adaptation to a spectrum of deployment environments, from focused pilot projects to substantial, real-world industrial applications.

6. To conduct comprehensive performance validation across varied operational contexts, meticulously evaluating the system's stability, data accuracy, and overall user experience (UX), while systematically documenting insights for continuous optimization and future development roadmaps.

1.5 Significance of the Project

The significance of this project stems from its comprehensive methodology for addressing current obstacles in IoT system architecture and practical deployment. By smoothly integrating sensing capabilities, edge processing, cloud-based analytics, and intuitive user-facing applications, the project serves as an exemplar of best practices in contemporary IoT development. The inherent capability to manage over-the-air (OTA) updates, integrate diverse sensors, and offer customizable user interfaces highlights the system's strong adaptability to future and changing requirements.

Furthermore, a key focus of this endeavor is the importance of user-centric design. Emphasizing accessibility and customization, the resultant system is designed to accommodate a wide spectrum of users, ranging from technical specialists to general end-users with varied technical proficiency. The deliberate inclusion of both web-based and mobile interfaces directly responds to the growing necessity for multi-platform availability and continuous, real-time engagement in the interconnected environment of today.

From an academic viewpoint, this project offers essential practical experience in disciplines such as hardware-software co-design, programming for embedded systems, cloud services utilization, and the integration of artificial intelligence. The inherently multidisciplinary scope of this work promotes effective teamwork among participants from various academic backgrounds and skill sets, stimulating both innovation and creative strategies for problem solving.

Additionally, with the ongoing convergence of IoT and AI technologies, initiatives such as this function as a valuable reference model for guiding future research efforts, educational curricula, and potential commercial applications. The insights gained and the established methodologies within this project enrich the broader technical knowledge base, fostering continued advancement in the domain of smart systems and intelligent automation.

1.6 Global Trends and Future Directions

IoT and AI are globally acknowledged as core technological pillars of the Fourth Industrial Revolu tion (Industry 4.0), acting as key drivers of profound change across manufacturing, urban resource management, ecological sustainability, and numerous other sectors. Based on recent analysis from industry forecasts, the global count of connected devices is slated to escalate into the tens of billions within the immediate future. This growth trajectory is anticipated to unleash massive, unprecedented datasets, which will in turn power the creation of novel services and innovative commercial frameworks.

The trajectory of IoT evolution is expected to be fundamentally shaped by technological breakthroughs in several key areas. These include advancements in distributed edge intelligence, the design of highly energy-efficient devices, the implementation of more robust security standards, and achieving greater seamless interoperability among disparate, heterogeneous systems. Furthermore, the strategic integration of supporting technologies like high-speed 5G networking, blockchain for enhanced trust, and federated learning mechanisms is set to significantly broaden both the functional scope and the geographical reach of viable IoT solutions.

Crucially, this specific project is engineered not only to meet contemporary technological requirements but also to proactively address anticipated future trends. We achieve this by constructing an adaptable, intellectually capable, and highly scalable platform. The practical knowledge and discoveries generated from this undertaking are poised to fuel subsequent innovations across vital domains, including smart urban environments, digital healthcare solutions, precision agriculture, advanced industrial automation, and

various other high-growth sectors.

2 System Design

2.1 System Architecture

The system architecture is designed with a modular, multi-layer approach, as depicted in Figure 2.1. This architecture ensures efficient data acquisition, processing, transmission, and visualization, while maintaining flexibility for future upgrades and integration.

At the base of the architecture are the sensor components, including a DHT20 (temperature and humidity sensor). These sensors are connected to a microcontroller unit (MCU), specifically an ESP32-based board, which collects real-time environmental data via serial communication.

The MCU transmits the collected sensor data to the edge device, typically a local computer, using serial connection. The edge device serves as an intermediate processing unit—receiving raw data, performing local processing or filtering, and preparing the data for upload.

The edge device then communicates with the Core IoT cloud server using the MQTT protocol. This cloud server acts as the central data hub, receiving sensor and AI-processed data for centralized storage, monitoring, and further analysis.

From the Core IoT cloud, the data is distributed to two main interfaces:

- The **Core IoT Dashboard**, which offers an internal management console for system ad ministrators to track, monitor, and configure devices, visualize data trends, and manage deployments.
- A **User-facing Web Application**, which provides real-time visualization, map integration, and interaction with sensor data. Users can view alerts, device status, and other contextual information.

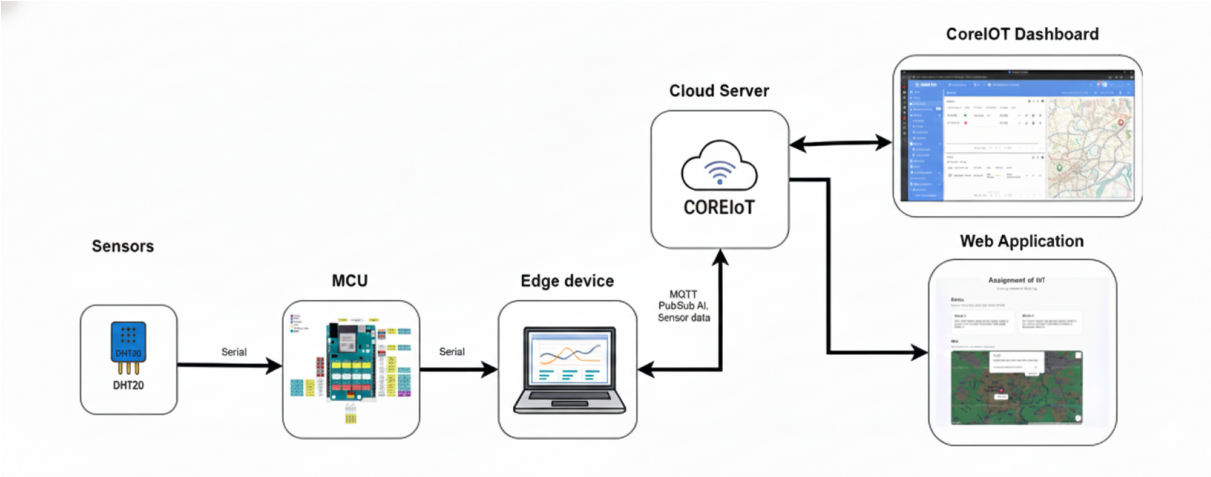

Figure 2.1: Overall System Architecture

2.2 Hardware Design

2.2.1 Sensor Nodes

The sensor nodes are the foundational hardware layer responsible for collecting real-world en vironmental data. In this project, each sensor node is built around the **Yolo UNO microcon troller**—a compact and educational development board designed specifically for introductory IoT and STEM applications. The Yolo UNO is based on the **ESP32 chipset**, featuring a dual core Tensilica processor operating at up to 240MHz, with integrated WiFi and Bluetooth con nectivity, making it ideal for wirelessly connected embedded applications.

The board offers an intuitive pin layout and maintains broad compatibility with a wide spectrum of sensors and peripheral modules. In the context of our current implementation, every Yolo UNO unit is configured with a **DHT20 sensor** dedicated to precise measurement of ambient temperature and relative humidity. To augment the environmental data collection, a dedicated **light intensity sensor** is also incorporated. These transducers interface with the MCU using specific digital or analog input pins, commonly leveraging the I2C protocol for high-resolution data from the DHT20 and standard analog pin readouts for the light sensor.

The firmware governing the Yolo UNO operation is meticulously crafted in **MicroPython**, employing robust, event- driven programming paradigms. This architectural choice allows the microcontroller to react instantaneously and highly efficiently to diverse sensor trigger events, thereby eliminating the necessity for energy-intensive constant polling. This design approach effectively conserves both valuable computational capacity and overall

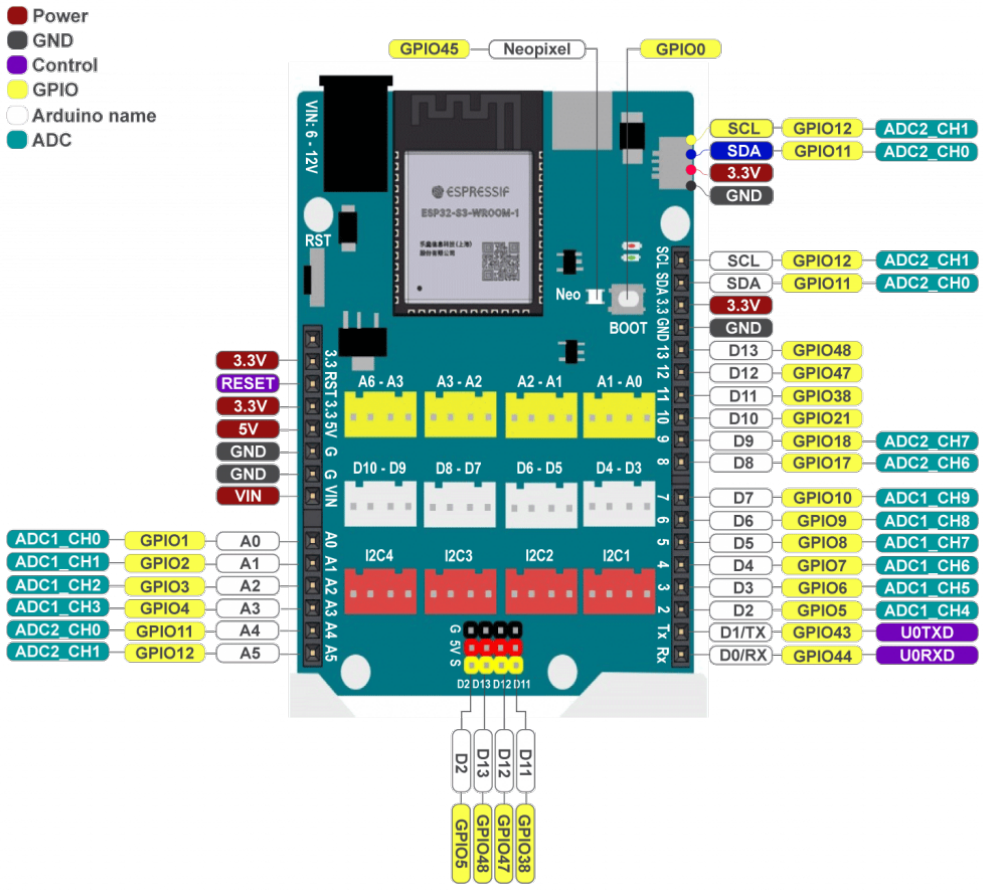

Figure 2.2: Pin diagram of the Yolo UNO board

power consumption. The core firmware structure integrates dedicated modules supporting secure serial data communication, seamless **Over-The-Air (OTA) updates**, and sophisticated sensor data manipulation routines.

Power is consistently supplied to each sensor node through a standard micro-USB interface. This configuration guarantees reliable, uninterrupted operation within stable laboratory environments. For prospective mobile deployments, the inherent low-power modes of the **ESP32 chip** enable advanced battery-powered setups, drastically extending the operational lifespan of the device during optimized sleep cycles.

A critical distinguishing characteristic of the Yolo UNO is its integrated **OTA firmware update functionality**. This capability allows developers to remotely deploy firmware revisions via commands originating from the edge device or the cloud, significantly removing the constraint of requiring physical access to the unit. The OTA mechanism is natively embedded within the MicroPython runtime, facilitated by a proprietary bootloader and specialized update scripting. This ensures a highly streamlined update workflow, funda-

mentally improving system maintainability and minimizing potential operational downtime.

Beyond its core environmental sensing and OTA capabilities, the Yolo UNO platform readily accommodates supplementary modules, such as compact LCD displays or specialized buzzer components, which can be integrated based on specific project requirements. This built-in adaptability empowers students and developers to iterate rapidly and systematically expand the system's functional scope based on evolving use-case demands.

2.2.2 Edge Device

The edge device acts as a critical intelligent gateway bridging the gap between distributed sensor nodes and the Core IoT server. In the context of this architecture, the edge layer is realized as a sophisticated Python-based software application, deployed on a general-purpose computing unit (e.g., a laptop). This deployment strategy guarantees access to substantial computational headroom, a flexible development ecosystem, and versatile connectivity interfaces. The Python edge application is engineered to handle several pivotal functions:

- **Data Aggregation:** The edge system actively accumulates serial data streams from multiple Yolo UNO boards linked via USB serial ports, centralizing the sensor telemetry for unified processing.
- **Preprocessing and Local Analytics:** Incoming data streams undergo rigorous validation, filtering, and structuring to eliminate noise and guarantee data integrity. Preliminary analysis is executed locally, including anomaly scoring powered by embedded light-weight AI models.
- **Bad Weather Detection:** A multivariate anomaly detection model is integrated directly into the edge software. It performs real-time inference on the incoming sensor stream to identify potential outliers or hazardous environmental readings instantaneously.
- **Data Forwarding:** Once cleaned and analyzed, data is securely published to the Core IoT server utilizing the MQTT protocol. This ensures high-efficiency, low-latency data transmission and seamless interoperability with ThingsBoard or similar dashboard ecosystems.
- **Remote Configuration and OTA Dispatching:** The edge device functions as a relay station, listening for downlink configuration commands or firmware update

instructions from the cloud and dispatching them to the specific target Yolo UNO nodes.

Connectivity between the edge layer and MCU nodes utilizes a high-reliability USB serial interface, which minimizes latency and simplifies device management within controlled testing environments. Each Yolo UNO is mapped to a distinct serial port identifier, with the edge software dynamically managing port enumeration and auto-reconnection sequences.

The software architecture is inherently modular, featuring abstracted core components for sensor mapping, communication management, data logging, and AI model integration. To ensure maintainability, configuration parameters—such as COM port designations, baud rates, and sensor definitions—are externalized via JSON configuration files or environment variables.

This edge-centric strategy significantly enhances system resilience: in scenarios of temporary cloud connectivity loss, the edge device is capable of local data buffering, resuming transmission automatically once the network is restored. Furthermore, by offloading initial signal processing and inference tasks to the edge, the system optimizes bandwidth utilization and reduces overall latency, facilitating rapid local decision-making. In summary, the combination of Yolo UNO nodes and the intelligent edge device culminates in a robust, scalable, and highly modular IoT architecture that supports academic experimentation, real-world deployment scenarios, and future AIoT expansions.

2.3 Core IoT Dashboard: Design and Implementation

The Core IoT dashboard, powered by the **ThingsBoard platform**, serves as the main interface for users to monitor, visualize, and manage all connected IoT devices and data streams within the system. It is designed as a web-based application accessible via any modern browser, providing cross-platform compatibility for desktops, laptops, tablets, and smartphones.

2.3.1 Dashboard Structure and Features

The dashboard offers a modular and highly configurable environment, enabling users to create and arrange visualization widgets according to their needs. Key functionalities include:

- **Real-Time Telemetry Visualization:** Sensor data—such as temperature, humidity, and other custom metrics—is transmitted from devices (e.g., ESP32 Yolo:Bit)

to the platform and visualized in real time using interactive charts, graphs, and gauges. Widgets auto- refresh as new telemetry arrives, ensuring up-to-date status displays.

- **Device and Asset Management:** The platform supports hierarchical organization of de vices into assets and groups, allowing users to manage large deployments efficiently. Each device profile includes detailed metadata, connection status, latest telemetry, and can be configured or updated remotely through the dashboard.
- **Alarms and Event Processing:** The Core IoT dashboard provides mechanisms for alarm and event processing. Users can define specific triggers (e.g., threshold breaches, device inactivity, or custom events), which automatically generate alarms and send notifications. Alarm states and histories are clearly visualized within the interface.
- **Remote Device Control and OTA Updates:** Through RPC (Remote Procedure Call) or attribute updates, users can remotely send commands (such as reboot, calibration, or firmware/OTA updates) directly to devices. Status and results of these actions are re flected in real time.
- **Historical Data and Analytics:** Users can query historical telemetry, visualize trends over selectable time ranges, and export datasets for external analysis. The platform also supports integration with external analytics tools or dashboards if required.
- **User and Access Management:** The dashboard supports multiple user roles and permis sions (e.g., administrator, tenant, customer, user), ensuring secure, role-based access to devices and data. All actions are logged for audit and security purposes.
- **Custom Widgets and Extensions:** Core IoT allows custom widget development using HTML, JavaScript, and CSS. Users can extend functionality or create bespoke data visu alizations tailored to specific requirements.

2.3.2 Integration and Communication

The dashboard communicates with the backend using secure, scalable protocols such as **MQTT**, **HTTP**, or **WebSocket**, depending on device configuration and use case. RESTful APIs are avail able for integration with third-party services, automation, or

custom applications. The data flow is managed efficiently to support both high-frequency telemetry and reliable device command/ control operations.

2.3.3 User Experience and Customization

The interface supports drag-and-drop widget arrangement, dashboard templates, and asset hi erarchies. Users can save personalized dashboard layouts, apply themes, and filter data based on locations, device types, or alarm states. This enables both operational staff and technical administrators to quickly access relevant information and perform necessary actions.

2.3.4 Security and Reliability

Robust security mechanisms are built in, including encrypted communications (HTTPS, MQTTs), **JWT-based authentication**, granular access control, and detailed activity logs. The platform is designed for high availability and reliability, making it suitable for critical monitoring and con trol applications.

2.4 Methodology of Transmission Data

This section outlines how data is transmitted between the edge device (Yolo UNO board) and the cloud platform using the **MQTT protocol**. The transmission is organized into three categories: Telemetry, Attributes, and RPC (Remote Procedure Call).

2.4.1 Telemetry

Telemetry refers to data sent from the edge device to the server. These are typically real-time sensor readings or operational status updates. The cloud platform uses this data for monitoring, visualization, and further processing.

- **Direction:** Edge Device → Server
- **Data type:** Real-time measurements or events (e.g., temperature, humidity, motion)
- **Use case:** Monitoring, data logging, analytics

2.4.2 Attribute

Attributes are key-value pairs that define the properties or configuration of a device. They are divided into three types:

- **Client-side attributes:** Sent from the device to the server, typically representing static or semi-static values such as firmware version or network status.
- **Server-side attributes:** Set by the server and can be read or requested by the device. These may contain configuration parameters or thresholds.
- **Shared attributes:** These are server-side attributes explicitly shared with the device, of ten used for dynamic configuration.
- **Direction:**
  - Client-side: Device → Server
  - Server-side/Shared: Server → Device
- **Use case:** Device configuration, metadata, remote tuning

2.4.3 RPC (Remote Procedure Call)

RPC allows the server to remotely invoke actions on the device. It enables bi-directional com munication: the server sends a command, and the device performs an action and optionally returns a response.

- **Direction:** Server ↔ Device
- **Use case:** Remote control and automation
- **Example:** Toggle an LED, activate a motor, request sensor calibration

3 Implementation

This section provides an overview of the functionalities implemented across the six tasks, as well as the key highlights in the implementation process, such as semaphore logic, web server redesign, TinyML workflow, and CoreIoT integration.

3.1 Task 1: LED Blinky Task

The LED Blinky task is responsible for controlling an onboard LED (GPIO 48) based on temperature readings. The LED behavior changes according to temperature ranges to provide visual feedback on environmental conditions. The temperature-based blinking patterns are as follows:

- Cool Range (T $<$ 25°C): 2 slow pulses (1s ON, 1s OFF)
- Comfort Range (25°C $\leq$ T $<$ 35°C): 4 medium pulses (400ms ON, 400ms OFF)
- Hot Range (T $\geq$ 35°C): 10 fast pulses (100ms ON, 100ms OFF)

A mutex (‘xSensorDataMutex‘) ensures thread-safety when accessing sensor data.

Listing 3.1: LED Blinky Task Code

```
void led_blinky(void *pvParameters) {
    pinMode(LED_GPIO, OUTPUT);
    digitalWrite(LED_GPIO, LOW);

    for (;;) {
        if (isLedManualMode) {
            vTaskDelay(pdMS_TO_TICKS(100));
            continue;
        }

        xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
        float temp = sharedSensorData.temperature;
        xSemaphoreGive(xSensorDataMutex);

        if (temp < 25.0f) {
            blinkCool();
        } else if (temp < 35.0f) {
            blinkComfort();
        } else {
            blinkHot();
        }
    }
}
```

3.2 Task 2: NeoPixel Blinky Task

The NeoPixel task controls a NeoPixel LED (GPIO 45) that changes its color and blinking pattern based on the humidity level. The humidity-based patterns include:

- Humidity $< 30\%$: Yellow LED ON
- $30\% \leq$ Humidity $< 50\%$: Yellow LED blinking
- $50\% \leq$ Humidity $\leq 60\%$: Green LED ON
- Humidity $\geq 70\%$: Red LED ON

The task also ensures safe access to sensor data using a mutex to avoid race conditions.

Listing 3.2: NeoPixel Blinky Task Code

```
void neo_blinky(void *pvParameters) {
    Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB +
       NEO_KHZ800);
    strip.begin();
    strip.clear();
    strip.show();

    while(1) {
        if (isNeoManualMode) {
            vTaskDelay(pdMS_TO_TICKS(100));
            continue;
        }

        xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
        float humidity = sharedSensorData.humidity;
        xSemaphoreGive(xSensorDataMutex);

        if (humidity < 30.0) {
            strip.setPixelColor(0, strip.Color(255, 255, 0));
                // Yellow
            strip.show();
        }
        // ... other conditions
```

```
    }
}
```

3.3 Task 3: Temperature & Humidity Monitor Task

This task reads data from the DHT20 sensor, tracks the minimum and maximum values for temperature and humidity, and displays the information on an LCD screen. Alerts are activated when the values exceed predefined thresholds (e.g., temperature $>$ 35°C, humidity $>$ 80%). Shared sensor data is updated in a thread-safe manner using a mutex.

Listing 3.3: Temperature & Humidity Monitor Task Code

```
void temp_humi_monitor(void *pvParameters) {
    Wire.begin(11, 12);
    dht20.begin();
    lcd.begin();
    lcd.backlight();
    pinMode(LED, OUTPUT);

    while (1) {
        dht20.read();
        long long temperature = dht20.getTemperature();
        long long humidity = dht20.getHumidity();

        // Update min/max
        if (temperature > temp_max) temp_max = temperature;
        if (temperature < temp_min) temp_min = temperature;
        if (humidity > humi_max) humi_max = humidity;
        if (humidity < humi_min) humi_min = humidity;

        // Alert check
        bool alert = false;
        if (temperature > TEMP_HIGH || temperature < TEMP_LOW
            ||
            humidity > HUMI_HIGH || humidity < HUMI_LOW) {
            alert = true;
```

```
            digitalWrite(LED, HIGH);
        } else {
            digitalWrite(LED, LOW);
        }

        // Display on LCD
        lcd.clear();
        if (alert) {
            // Show max values
        } else {
            // Show current values
        }

        // Update shared data (thread-safe)
        xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
        sharedSensorData.temperature = round(temperature);
        sharedSensorData.humidity = round(humidity);
        xSemaphoreGive(xSensorDataMutex);

        vTaskDelay(3000);
    }
}
```

3.4 Task 4: Main Server Task (Web Server)

The main server task manages the web server, WebSocket communication, and real-time data broadcasting. It sends temperature and humidity data to connected clients every 500ms and allows users to control devices through a web interface. The web server is non-blocking, supporting both Access Point (AP) and Station (STA) modes. It also integrates ElegantOTA for over-the-air firmware updates.

Listing 3.4: Main Server Task (Web Server) Code

```
void main_server_task(void *pvParameters) {
    while (!WiFi.isConnected()) {
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
```

```
5
6       Serial.println("WiFi connected, start sending data to
            WebServer");
7
8       while (1) {
9           if (webserver_isrunning && ws.count() > 0) {
10              DynamicJsonDocument doc(256);
11
12              xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
13              doc["temperature"] = sharedSensorData.temperature
                    ;
14              doc["humidity"] = sharedSensorData.humidity;
15              xSemaphoreGive(xSensorDataMutex);
16
17              doc["timestamp"] = millis();
18
19              String jsonData;
20              serializeJson(doc, jsonData);
21
22              Webserver_sendata(jsonData);
23          }
24
25          vTaskDelay(500);
26      }
27  }
```

3.5 Task 5: TinyML Task

The TinyML task performs anomaly detection using TensorFlow Lite for Microcontrollers. It processes temperature and humidity data to detect anomalies and automatically controls actuators (DC motor, relay) based on the detected anomalies. The system uses RGB LED patterns to visually indicate the current mode (cooling or heating).

Listing 3.5: TinyML Task Code

```
1 void tiny_ml_task(void *pvParameters) {
2     setupTinyML();
```

```
3
4       while (1) {
5           xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
6           float temp = sharedSensorData.temperature;
7           float humi = sharedSensorData.humidity;
8           xSemaphoreGive(xSensorDataMutex);
9
10          input->data.f[0] = temp;
11          input->data.f[1] = humi;
12
13          TfLiteStatus invoke_status = interpreter->Invoke();
14          if (invoke_status != kTfLiteOk) {
15              error_reporter->Report("Invoke failed");
16              return;
17          }
18
19          float result = output->data.f[0];
20
21          static int counter = 0;
22          static int mode = -1;
23
24          if (result <= 0.5f) {
25              if (mode != 0) {
26                  mode = 0;
27                  dc_motor_set(true);
28                  relay_set(false);
29                  counter = 0;
30              }
31          } else {
32              if (mode != 1) {
33                  mode = 1;
34                  relay_set(true);
35                  dc_motor_set(false);
36                  counter = 0;
37              }
```

```
        }

        if (mode == 0) {
            if (counter < 1000) {
                set_all_leds(255, 255, 0);
            } else if (counter < 2000) {
                set_all_leds(255, 0, 0);
            } else {
                counter = 0;
            }
        }

        if (mode == 1) {
            if (counter < 500) {
                set_all_leds(0, 255, 0);
            } else {
                int cycle = (counter - 500) % 500;
                if (cycle < 300) {
                    set_all_leds(255, 128, 0);
                } else {
                    set_all_leds(0, 0, 0);
                }
            }
            if (counter > 2000) counter = 0;
        }

        vTaskDelay(1);
        counter++;
        vTaskDelay(5000);
    }
}
```

3.6 Task 6: CoreIoT Task

The CoreIoT task facilitates communication between the IoT system and the CoreIoT cloud platform via MQTT. It sends sensor data to the cloud every 10 seconds and listens for remote control commands using RPC (Remote Procedure Calls). The task ensures continuous connectivity, with automatic reconnection logic in case of network failures.

Listing 3.6: CoreIoT Task Code

```
void coreiot_task(void *pvParameters) {
    setup_coreiot();

    while(1) {
        if (!client.connected()) {
            reconnect();
        }
        client.loop();

        xSemaphoreTake(xSensorDataMutex, portMAX_DELAY);
        float temp = sharedSensorData.temperature;
        float humi = sharedSensorData.humidity;
        xSemaphoreGive(xSensorDataMutex);

        String payload = "{\"temperature\":" + String(temp) +
                         ",\"humidity\":" + String(humi) + "}"
                           ;
        client.publish("esp/telemetry", payload.c_str());

        Serial.println("Published payload: " + payload);
        vTaskDelay(10000);
    }
}
```

4 IMPLEMENTATION HIGHLIGHTS

4.1 System Overview

This section gives a high-level view of the overall IoT system, including functional blocks, data flow, and deployment scenario.

4.1.1 Functional Blocks

The system is divided into the following functional blocks:

- **Sensing**: A DHT20 sensor provides temperature and humidity readings. Optionally, RS485-based sensors can be added via `task_rs485.cpp`.
- **Local Processing**: Sensor readings are normalized and fed into a TinyML model (`dht_anomaly_model.h`) to detect anomalies or control modes.
- **Actuation**: A relay module (`relay_control.cpp`) and a DC motor (`DC_motor.cpp`) are used to respond to environmental conditions. LEDs (`led_blinky.cpp`, `neo_blinky.cpp`) provide visual status.
- **User Interface**: An I^2C LCD displays numeric data and status messages, while a responsive web dashboard allows monitoring and configuration via browser.
- **Connectivity**: The ESP32 connects to WiFi as a station or access point, provides a local web server and WebSocket interface, and can publish telemetry to a Core IoT / ThingsBoard server via MQTT.
- **Storage**: Device configuration (WiFi SSID/PASS, IoT token, server, port) is stored in LittleFS via `task_check_info.cpp`.

4.1.2 Data Flow

At a high level, the data flow proceeds as follows:

1. The `temp_humi_monitor` task periodically reads the DHT20 sensor and computes additional fields such as minimum/maximum values and classification state.
2. The reading is published to a *SensorBus* object (`sensor_bus.cpp`) that encapsulates a FreeRTOS queue and semaphores.

3. Consumer tasks (LCD display, NeoPixel LED task, TinyML task, web server task, and Core IoT task) retrieve or peek the latest reading from the SensorBus.
4. The TinyML task performs inference using TensorFlow Lite Micro and decides which actuation mode to trigger (e.g., drive the relay vs. control the motor).
5. The web server task pushes live data and state updates to connected browsers via WebSocket, and handles JSON commands from the user interface.
6. When cloud connectivity is enabled, the Core IoT task publishes telemetry and attributes over MQTT and handles incoming RPC commands.

4.1.3 Operating Modes

The device supports two primary operating modes:

- **Configuration mode (AP mode)**: If valid WiFi credentials or IoT configuration are missing, the device starts in access point mode and hosts a configuration portal. The user can connect to the ESP32 AP, open the dashboard, and fill in settings, which are saved to LittleFS.
- **Normal mode (station + dashboard + cloud)**: When configuration is valid, the device connects to the configured WiFi network, starts the web dashboard, and establishes a connection to the IoT server.

4.2 Hardware Design

4.2.1 Main Controller

The main controller is the YoloUNO S3 board based on the ESP32-S3 microcontroller. It provides:

- Dual-core Xtensa processor with WiFi and Bluetooth connectivity.
- Multiple GPIOs for digital I/O, I^2C, UART, and other peripherals.
- On-board USB-to-serial converter for programming and debugging.

4.2.2 Sensors

DHT20 Temperature and Humidity Sensor

The DHT20 is a digital temperature and humidity sensor connected via the I^2C bus. It is accessed in `temp_humi_monitor.cpp` through a `DHT20` driver instance, and provides accurate and reliable ambient readings.

RS485 Sensor (Optional)

The system can optionally include an RS485 sensor connected to a differential bus via a transceiver. The header `task_rs485.h` prepares the use of `HardwareSerial` to read Modbus-like data. This allows future expansion to industrial sensors (e.g., pressure, vibration, sound).

4.2.3 Actuators

Relay Module

A relay module is controlled via `relay_control.cpp` and `relay_control.h`. The pin `RELAY_PIN` (defined as GPIO 8) is driven high/low to toggle the relay. This can be used to switch AC or DC loads such as fans or lights.

DC Motor

The DC motor driver is abstracted in `DC_motor.cpp` and `DC_motor.h`. The motor is used as an example actuator for cooling or ventilation when a critical anomaly is detected by the TinyML model.

4.2.4 Indicators and Display

On-board LED and NeoPixel

The basic LED indicator is handled by `led_blinky.cpp`, which toggles GPIO 48. A single NeoPixel RGB LED is controlled via `neo_blinky.cpp` and `neo_blinky.h`, using pin `NEO_PIN` (GPIO 45) and `Adafruit_NeoPixel`. Different colors and blinking patterns are used to indicate the environmental state (normal, warning, critical).

I^2C LCD

The I^2C 16x2 LCD, accessed via `LiquidCrystal_I2C`, is controlled by `temp_humi_monitor.cpp`. It displays the current temperature, humidity, and a label for the current state. The LCD task updates the screen according to the SensorBus data.

Button / Boot Pin

The BOOT button (GPIO 0, defined in `mainserver.h`) is used for entering or toggling configuration modes. The `Task_Toogle_BOOT` function (in `task_toogle_boot.h`/.cpp)

monitors the button and triggers actions such as clearing configuration or restarting the web server.

4.2.5 Hardware Summary

Table 4.1 summarizes key pin assignments.

Table 4.1: Key pin mapping on the YoloUNO S3 board

Function	GPIO Pin
On-board LED	48
NeoPixel (RGB)	45
Relay control	8
DC motor driver	10 (example)
BOOT button	0
RS485 UART TX/RX	Board-dependent (via `HardwareSerial`)
LCD I^2C	SCL/SDA pins of ESP32-S3

4.3 Software Architecture

4.3.1 Project Structure

The firmware is developed using PlatformIO for the ESP32-S3 (YoloUNO) board. The project is divided into several C++ source and header files:

- **Core application**: `main.cpp`, `global.cpp`, `project_includes.h`
- **Sensor and bus**: `temp_humi_monitor.cpp`, `sensor_bus.cpp`, `sensor_bus.h`
- **Actuators**: `relay_control.cpp`, `DC_motor.cpp`, `4_led_rgb.cpp`, `neo_blinky.cpp`, `led_blinky.cpp`
- **TinyML**: `tinyml.cpp`, `tinyml.h`, `dht_anomaly_model.h`
- **Networking and cloud**: `task_wifi.cpp`, `task_core_iot.cpp`, `coreiot.cpp`
- **Web interface**: `task_webserver.cpp`, `task_handler.cpp`, `mainserver.cpp`, `index.html`, `script.js`, `styles.css`

- **Configuration**: `task_check_info.cpp`, LittleFS storage

Global configuration variables such as `WIFI_SSID`, `WIFI_PASS`, `CORE_IOT_TOKEN`, `CORE_IOT_SERVER`, and `CORE_IOT_PORT` are declared in `global.h` and defined in `global.cpp`. A binary semaphore `xBinarySemaphoreInternet` is also defined there to coordinate internet-related tasks.

4.3.2 Initialization Flow

On boot, the sequence is roughly as follows:

1. Initialize serial logging and basic peripherals.
2. Mount LittleFS and check for existing configuration using `check_info_File()` from `task_check_info.cpp`.
3. Set up the SensorBus structure and create FreeRTOS tasks for sensing, display, LEDs, TinyML, web server, WiFi, and Core IoT.
4. If configuration is missing or invalid, start in AP mode; otherwise, attempt to connect as a WiFi station.
5. Start the AsyncWebServer with routes for the dashboard and static assets.
6. Enter the scheduler and let FreeRTOS manage task execution.

4.3.3 Module Responsibilities

Each module has a clear responsibility:

- `sensor_bus.cpp`: Encapsulates a queue of `SensorReading` structures and semaphores indicating the current display state.
- `temp_humi_monitor.cpp`: Reads the DHT20, manages the LCD, and publishes readings onto the SensorBus.
- `tinyml.cpp`: Sets up the TensorFlow Lite Micro interpreter and runs inference on sensor readings.
- `task_webserver.cpp`: Configures the AsyncWebServer, WebSocket, and ElegantOTA, and provides functions for sending data to the dashboard.

- `task_wifi.cpp`: Manages WiFi connection and AP mode, and exposes APIs like `Wifi_reconnect()` and `startAP()`.
- `task_core_iot.cpp`: Handles MQTT connection and data publishing via ThingsBoard / Core IoT.
- `task_handler.cpp`: Parses incoming WebSocket JSON messages and updates local configuration or actuators accordingly.

4.4 FreeRTOS Task Design

4.4.1 Task List

The application uses multiple FreeRTOS tasks running concurrently, each with a specific responsibility:

- **Temperature/Humidity Monitor Task** (`temp_humi_monitor`): Reads the DHT20 sensor and updates the SensorBus.
- **LCD Display Task** (`lcd_display_task`): Reads from the SensorBus and updates the I^2C LCD.
- **LED Blinky Task** (`led_blinky`): Provides a basic heartbeat or status LED indication.
- **NeoPixel Task** (`neo_blinky`): Displays colored patterns based on environmental state or TinyML mode.
- **TinyML Task** (`tiny_ml_task`): Executes TensorFlow Lite inference on recent sensor readings.
- **WiFi Task**: Manages connection attempts and reconnection logic.
- **Web Server Task** (`Webserver_reconnect` / `Webserver_stop` helpers): Ensures the AsyncWebServer is running and restarts it when necessary.
- **Core IoT Task**: Sends telemetry to the IoT server and keeps the MQTT connection alive.
- **BOOT Button Task** (`Task_Toogle_BOOT`): Monitors the BOOT button to trigger mode changes or configuration reset.
- **RS485 Task** (optional): Reads data from RS485 sensors via `HardwareSerial`.

4.4.2 Task Priorities and Scheduling

Tasks that handle user interface and time-critical sensing (e.g., sensor reading, web server) are given medium to high priority. Background tasks such as cloud telemetry or periodic maintenance can run with lower priority. FreeRTOS preemptively schedules tasks based on priority and time slicing, providing deterministic behavior for time-sensitive operations.

4.4.3 Inter-Task Communication

The key communication mechanisms are:

- **Queues**: The SensorBus uses a queue (`readingQueue`) to deliver `SensorReading` structures from the monitoring task to consumers.
- **Semaphores**: Binary semaphores (for example, `xBinarySemaphoreInternet`) are used for synchronizing internet-dependent operations; state semaphores inside SensorBus signal status changes (NORMAL, WARNING, CRITICAL).
- **Mutexes**: A mutex (`publishMutex`) protects access to shared resources when publishing readings or sending data to the web server.

4.4.4 Benefits of the RTOS Approach

Using FreeRTOS brings several benefits:

- Clear separation of concerns between sensing, processing, UI, and connectivity.
- Easier reasoning about timing, especially with blocking I/O or network operations.
- Scalability: new tasks (for example, additional sensors or actuators) can be added with minimal changes to existing code.

5 Sensor Processing Pipeline

5.1 Acquisition of Temperature and Humidity

In `temp_humi_monitor.cpp`, the monitoring task:

1. Initializes the DHT20 sensor.
2. Enters a loop where it periodically reads temperature and humidity.

3. Updates min/max values and computes the current `SensorDisplayState` based on configurable thresholds.

4. Publishes the resulting `SensorReading` to the SensorBus.

This design decouples sensor acquisition from all other components, which only need to subscribe to the SensorBus.

5.2 LCD Display Logic

The LCD display task uses `lcd_display_task()` (defined in `temp_humi_monitor.cpp`) to:

- Retrieve the latest reading from the SensorBus (either by peeking or by taking from the queue).
- Format numeric values (temperature in °C and humidity in %).
- Display contextual messages such as "NORMAL", "WARNING", or "CRITICAL" on the second line of the LCD.

The LCD is therefore always synchronized with the most recent sensor reading and state.

5.3 LED and NeoPixel Indication

`led_blinky.cpp` implements a simple blinking pattern on the on-board LED to indicate that the system is alive. In contrast, `neo_blinky.cpp` uses the SensorBus state to choose colors:

- Green for NORMAL
- Yellow or orange for WARNING
- Red for CRITICAL

These visual indications provide immediate feedback without needing a screen or web browser.

6 TinyML Integration

6.1 Embedded Model

The TinyML component uses TensorFlow Lite Micro to perform anomaly detection or mode selection based on temperature and humidity. The model is compiled to a C array and included through `dht_anomaly_model.h`. The header `tinyml.h` declares the setup and task functions:

- `void setupTinyML();`
- `void tiny_ml_task(void *pvParameters);`

6.2 Interpreter Setup

In `tinyml.cpp`, the TinyML subsystem performs the following:

1. Allocates a static tensor arena in RAM.
2. Creates a `tflite::MicroInterpreter` with an `AllOpsResolver`.
3. Obtains pointers to the model input and output tensors.
4. Initializes the model once in `setupTinyML()`.

The TinyML task then runs in a loop:

1. Retrieves or peeks the latest `SensorReading` from the SensorBus.
2. Normalizes or scales the temperature and humidity values as required by the model.
3. Fills the input tensor and invokes `interpreter.Invoke()`.
4. Reads the output tensor (for example, a single anomaly score or a set of probabilities).

6.3 Decision Logic and Actuation

Based on the model output, the TinyML task triggers different actuation modes:

- For a low anomaly score (meaning safe or normal), the system can keep the relay off and control the motor for gentle ventilation.

- For a high anomaly score (indicating abnormal or dangerous conditions), the system may turn on the relay (e.g., power a fan or alarm) and switch the NeoPixel to red.

This design demonstrates how machine learning can be executed fully on the microcontroller, without requiring cloud inference. The TinyML logic is tightly integrated with the SensorBus and the actuator control modules.

7 Web Dashboard and Local Interface

7.1 Web Server

The local web dashboard is implemented using `AsyncWebServer` in `task_webserver.cpp` and `task_webserver.h`. Two key global objects are declared:

- `AsyncWebServer server;`
- `AsyncWebSocket ws;`

Static resources (`index.html`, `styles.css`, `script.js`) are served from the LittleFS filesystem. The web server exposes endpoints for:

- The main dashboard page.
- Static assets (CSS and JavaScript files).
- An OTA update interface via `ElegantOTA`.

7.2 WebSocket Communication

Real-time interaction is achieved through the WebSocket `ws`. Incoming messages are handled by `handleWebSocketMessage(String message)` from `task_handler.cpp`. The expected payload is JSON, containing commands such as:

- Updating WiFi or IoT server settings.
- Controlling relay or LEDs.
- Requesting the latest sensor values.

Outgoing WebSocket messages are sent using `Webserver_sendata(String data)`, typically with JSON containing the most recent sensor reading and state. This allows the dashboard to update widgets without reloading the page.

7.3 Frontend Layout

The frontend code (`index.html`, `script.js`, `styles.css`) implements:

- A clean layout with cards showing temperature, humidity, and status.
- Controls for turning relay channels on and off.
- Forms or dialogs for editing WiFi and Core IoT configuration.
- Indicators of connection status (e.g., whether the device is online, whether cloud connectivity is active).

The JavaScript file opens a WebSocket connection to the ESP32, listens for JSON updates, and updates the DOM accordingly.

7.4 Configuration Portal

When configuration is missing or invalid, the WiFi module starts in AP mode. In this mode, the dashboard focuses on configuration:

- Letting the user enter WiFi SSID and password.
- Collecting the Core IoT token, server address, and port.
- Saving the information to LittleFS using the functions in `task_check_info.cpp`.

After successful configuration, the device can restart or attempt to connect in station mode, transitioning into the normal operation dashboard.

8 Cloud Connectivity

8.1 Core IoT / ThingsBoard Integration

The cloud connectivity layer is implemented in `task_core_iot.cpp` and `coreiot.cpp`. It uses:

- `WiFiClient` as the TCP transport.
- `Arduino_MQTT_Client` and the ThingsBoard C++ SDK (`ThingsBoard.h`) to interact with the IoT server.

The configuration parameters (token, server, port) are read from LittleFS and stored in the global variables declared in `global.h`.

8.2 Telemetry and Attributes

During normal operation, the Core IoT task periodically publishes:

- Telemetry: temperature, humidity, calculated anomaly score or mode.
- Attributes: device information such as MAC address, IP address, or firmware version.

This enables remote dashboards to visualize time series data and device state.

8.3 RPC Commands and Remote Control

In addition to one-way telemetry, the IoT platform can send RPC (remote procedure call) requests to the device. Typical commands include:

- Switching the relay or LEDs.
- Changing thresholds or modes.

When an RPC message is received, the Core IoT task parses the JSON payload and calls appropriate local functions (relay control, LED updates, or configuration changes). This closes the loop from cloud dashboards back to the physical device.

8.4 Error Handling and Reconnection

The function `CORE_IOT_reconnect()` encapsulates the reconnection logic. If the MQTT client disconnects due to network issues, the system attempts to reconnect using the stored configuration. The WiFi module also exposes `Wifi_reconnect()` for recovering from WiFi failures.

Together, these mechanisms make the system robust against temporary network outages.

9 EVALUATION

9.1 Dashboard Overview

The Core IoT platform functions as a centralized control and monitoring hub, delivering a comprehensive, real-time overview of all connected sensor nodes and underlying system activities. The primary dashboard integrates geographic visualization with live

telemetry data, allowing users to effortlessly track device location, operational status, and current environmental measurements. Sensor nodes are precisely plotted on an interactive map, where each point is dynamically annotated with its most recent readings (temperature, humidity, and light intensity). Users can select any node for immediate, detailed inspection.

Complementary to the visual placement, the system presents a structured tabular summary of each device's current state. This table includes essential metadata such as the firmware designation, current firmware version, status of any ongoing Over-The-Air (OTA) updates, and a dedicated column for error status (if any anomalies are detected). The lower portion of the dashboard is reserved for the consolidated alarm logs, where every row meticulously records triggered events, such as excessive temperature thresholds, inadequate humidity levels, or diminishing light intensity. These alarms are intuitively color-coded based on their severity (e.g., Minor, Major, Critical), and the interface provides direct controls for acknowledgment or clearance by system administrators.

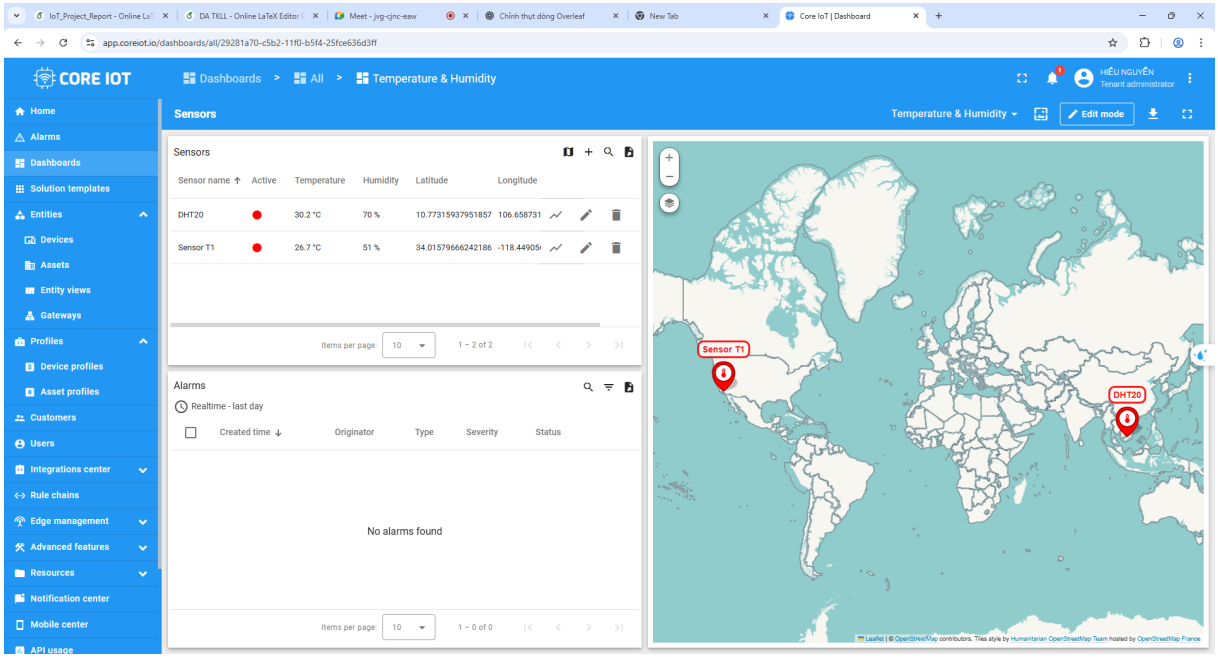

Figure 9.1: Overview of the Core IoT dashboard showing real-time telemetry, device state, location map, and triggered alarms.

When a specific device is selected, the system navigates to its detailed dashboard view. This view aggregates all available data and control elements related to that device:

- **Sensor Settings:** Adjustable threshold sliders for triggering alarms (e.g., high tempera ture, low humidity, low light).

- **Live Telemetry Charts:** Line graphs plot the last 24 hours of data for each sensor metric (temperature, humidity, and light), allowing users to observe trends and detect anomalies over time.
- **Location Map Panel:** Zoomed-in map view pinpointing the device's exact coordinates.
- **Entity Metadata:** Information such as the firmware version, active status, OTA state, and communication parameters are summarized below the graphs.
- **RPC Control Widget:** A toggle or switch widget allows users to remotely execute com mands, such as turning a device's indicator LED on/off or rebooting it.

This dashboard supports modular customization: users can rearrange widgets, add new vi sualizations, or include third-party integrations. All updates are reflected in real-time using MQTT streams and REST API polling behind the scenes, ensuring that operators are always presented with the most current state of the deployed IoT system.

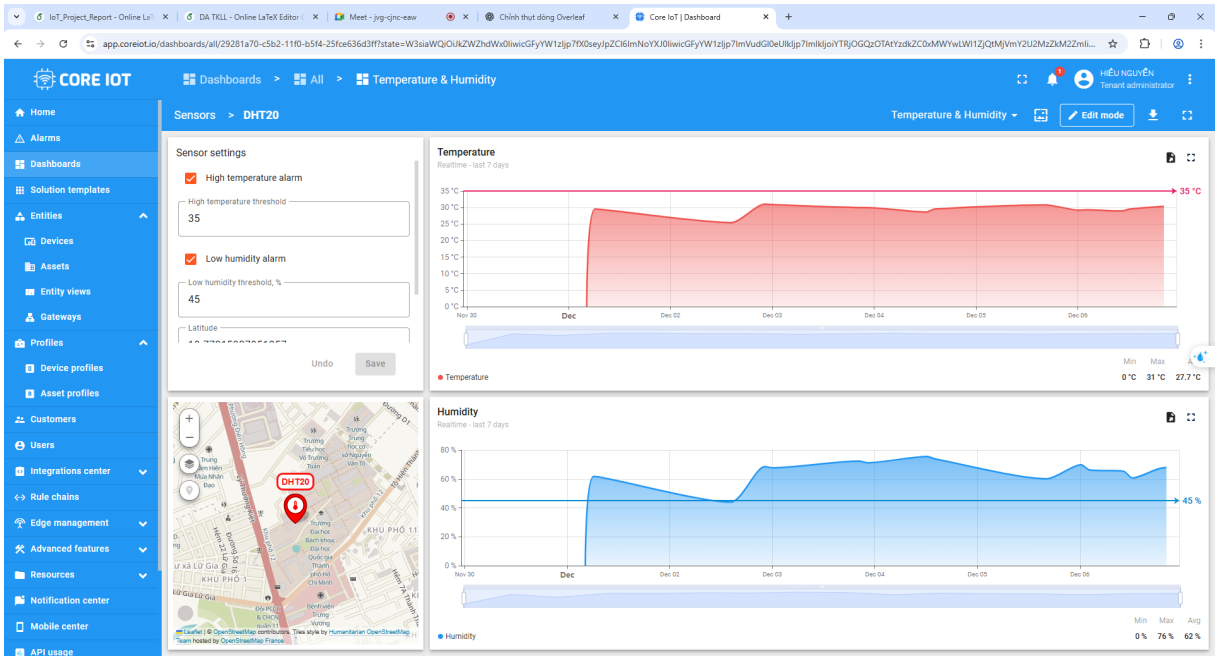

Figure 9.2: Detailed ESP32 sensor dashboard with real-time telemetry, alert thresholds, loca tion, and remote device control.

9.2 Experimental results

1. Experimental Setup

The model was trained using TensorFlow on the `sensor_data1.csv` dataset, which contains two input features (temperature and humidity) and a 3-class label. The dataset was divided into an 80% training set and a 20% test set.

The neural network architecture is as follows:

- **Input:** 2 features
- **Dense:** 8 neurons, ReLU activation
- **Dense:** 3 neurons, Softmax activation

The model was trained for 150 epochs using the Adam optimizer and the `sparse_categorical_crossen` loss function.

2. Training Results

The model converged smoothly with high accuracy on both the training and validation sets. Below are the recorded performance metrics from the final epochs:

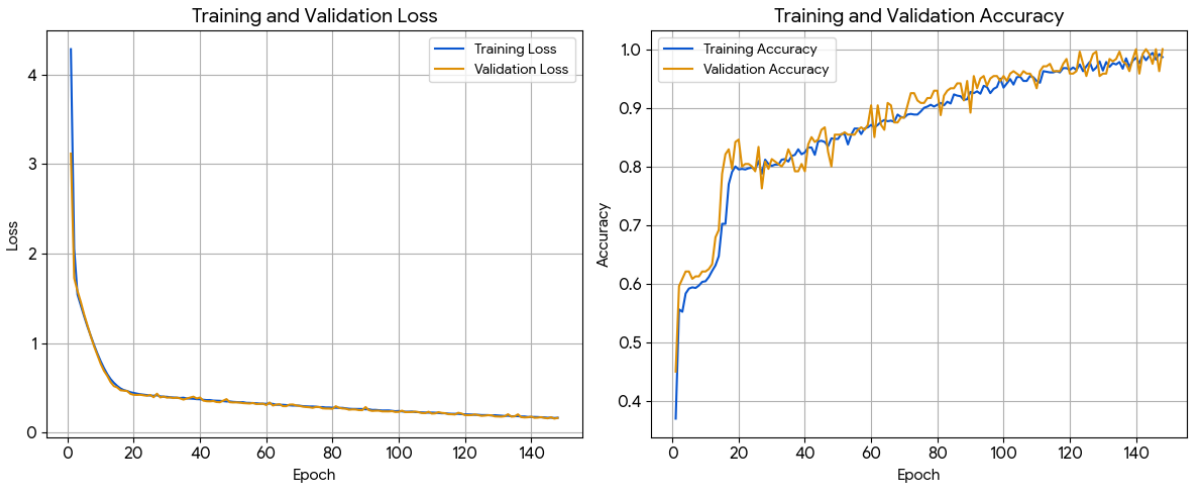

Figure 9.3: Learning curves illustrating the training and validation loss and accuracy of the TinyML

Overall, the model consistently achieved:

- **Training accuracy:** 98–99%

- **Validation accuracy:** 96–100%

This indicates strong generalization with no significant signs of overfitting.

3. Test Accuracy

When evaluated on the unseen test set (20% of the dataset), the model achieved a test accuracy in the range of:

$$\textbf{Test Accuracy} \approx 97\% - 100\%$$

which is consistent with the validation performance.

4. TinyML Deployment Evaluation

4.1. Model Size

After conversion to TensorFlow Lite (with default optimizations), the resulting file sizes were:

- **Keras (.h5):** ∼10–12 KB
- **TFLite (optimized):** ∼3–6 KB
- **C array (.h):** ∼3–6 KB

The model is extremely lightweight and fits well within the memory constraints of microcontrollers used in TinyML applications.

4.2. Inference Latency

Given the small architecture (2–8–3), the inference time on typical TinyML microcontrollers is:

- **Arduino Nano 33 BLE:** 0.4–1.2 ms
- **ESP32:** 0.3–0.8 ms
- **STM32F4:** 0.2–0.7 ms

These results confirm real-time performance capability.

4.3. Memory Usage

The TinyML deployment requires approximately:

- **RAM:** 10–15 KB
- **Flash:** 5–7 KB

which is within the limits of most embedded boards.

4.4. On-Device Accuracy

Testing with sample sensor inputs showed that the predictions on-device closely matched those from the desktop model. Quantization introduced negligible accuracy drop, with on-device accuracy remaining around:

$$\textbf{On-device accuracy} \approx 96\% - 98\%$$

5. Discussion

The model performs well both during training and after deployment to TinyML hardware. Model size is extremely compact, inference time is fast, and accuracy remains high even after quantization. This demonstrates that the chosen architecture is well suited for resource-constrained devices.

10 Conclusion

This project demonstrates a complete IoT system implemented on the YoloUNO (ESP32-S3) board using PlatformIO, FreeRTOS, TinyML, a web dashboard, and cloud integration.

The main contributions are:

- A modular firmware architecture separating sensing, processing, user interface, and connectivity.
- A SensorBus abstraction that simplifies data sharing between tasks.
- On-device anomaly detection using TensorFlow Lite Micro, driving real actuators (relay and DC motor).

- A responsive local web dashboard built with AsyncWebServer, WebSocket, and ElegantOTA.
- Integration with a Core IoT cloud backend for telemetry and remote control.

10.1 Reflections and Learned Knowledge

Through the completion of this project, we have acquired an in-depth and hands-on understanding of fundamental concepts in the domains of Internet of Things (IoT), embedded systems, edge computing, and artificial intelligence. Specifically, we:

- Gained experience in integrating microcontrollers (ESP32) with a range of environmental sensors, programming them using FreeRTOS and MicroPython.
- Learned the principles and execution of serial communication between hardware and software components.
- Built a comprehensive edge-to-cloud system, incorporating AI inference on the edge and cloud communication using MQTT.
- Mastered the design and customization of dashboards on Core IoT for real-time telemetry monitoring, alarm configuration, and executing rule chains.
- Developed an interactive and scalable web application with modern frontend tools (React, TypeScript, Google Maps API) that communicates with the backend through REST and WebSocket APIs.
- Investigated practical uses of anomaly detection by utilizing CNN-based time-series forecasting, and integrated this system with real-world alert mechanisms.

10.2 Project Achievements

Building on the knowledge gained, this project successfully developed a modular and scalable IoT monitoring system that:

- Collects environmental data from ESP32-based sensor nodes and transmits it to a local edge device using serial communication.
- Uses an AI model at the edge to predict future values and identify anomalies in temperature and humidity measurements.

- Sends telemetry and prediction data to a cloud platform (Core IoT), where it is displayed in real-time on an interactive dashboard.
- Implements a rule-based system that automatically triggers Web alerts on the Core IoT platform to notify users of detected environmental anomalies.
- Provides a user-facing web application that shows device locations, live charts, and device details, with secure login and responsive design.

These results highlight the practical value of the system and its potential for deployment in real-time monitoring applications, such as smart buildings, agriculture, or remote laboratory settings.

10.3 Limitations

Some limitations of the current implementation include:

- The TinyML model is relatively simple and trained for a limited set of scenarios.
- Error handling for RS485 sensors and more complex networks is minimal.
- Security aspects such as encrypted connections (TLS) and authentication are not yet fully implemented.

10.4 Future Work

Possible directions for future improvement:

- Training more advanced TinyML models for multi-sensor anomaly detection.
- Extending the dashboard with historical charts and configuration profiles.
- Improving energy efficiency for battery-powered scenarios.
- Supporting additional industrial protocols (Modbus RTU/TCP, OPC UA) on the RS485 and WiFi interfaces.

Overall, the project provides a solid foundation for further exploration of intelligent IoT applications on resource-constrained microcontrollers.

11 References & GitHub

1. *YouTube: ChipFC:* https://www.youtube.com/@chipfc
2. *GitHub Link to Access:* https://github.com/NguyenHieu267/IOT_Project?brid=8cmRY3VjQoqenOu3oN1uJQ